Designing an Interactive Educational Framework for LZSCC.111

Andrei Florea (39603784) • Supervisor: Tobias Meggendorfer • 29th October 2026

Abstract

Students on LZSCC.111 Software Development learn to program in C, where program state is hidden and mistakes often show up only as a wrong number or a crash. This project will build a framework of interactive C exercises that increase in difficulty. In each exercise the student implements a single function. Provided scaffolding handles input and output, and a Python visualiser shows what the code does step by step, such as the bars of an array being searched or a robot moving through a maze. Student code runs inside a C harness that records each step as an event trace and catches crashes, infinite loops and out-of-bounds access. The framework will be evaluated technically and through a small study with students, comparing the exercises with and without the visualiser. The expected outcome is a reusable set of tasks for LZSCC.111, along with indicative evidence on whether visual feedback improves novice understanding and engagement.

1 Introduction

LZSCC.111 introduces students to programming in C. For a beginner, C gives little visible feedback [1]. A loop that searches an array or a function that moves a robot produces at most a line of printed output, and the student cannot see how the program reached its result [2]. Reading input, parsing it and formatting output also take effort unrelated to the concept being taught, so beginners spend their time on scanf and printf instead of the algorithm. Research on algorithm visualisation has found that showing execution helps most when students interact with the visualisation rather than only watch it [3], [4]. This project applies that idea to LZSCC.111. Students write only the core logic of each task in C, and a Python visualiser shows its effect. The rest of this proposal is organised as follows. Section 2 reviews related work and the gap this project addresses. Section 3 sets out the aims and objectives, followed by the methodology. Section 4 describes the programme of work, Section 5 relates to the resources required and last but not least Section 6 reveals the risks and contingencies.

2 Background and Related Work

Novices learn to program by building a notional machine, a mental model of what happens when code runs [2], and an incomplete model causes many misconceptions [5]. The model is hard to build in C, where state is hidden and errors often appear only as a wrong number or a crash. Smith and Webb showed that visualisation helps novice C programmers [1], although their tool displayed generic memory rather than the concept being taught. Here, each task is visualised in its own terms, such as array bars or a robot in a maze.

Hundhausen et al. found that visualisations work best when students engage with them actively rather than just watching [3], and Naps et al. formalised this as an engagement taxonomy [4]. Classic algorithm animations replay pre-built code and so keep students at the “viewing” level. In this framework the animation comes from the student’s own code, which places them at the “constructing” level.

General-purpose tools such as Jeliot 3 [6] and Online Python Tutor [7] visualise arbitrary programs, and Python Tutor also lets students step backwards. What they display is limited to

variables, frames and pointers, not what an algorithm is doing, and most of them support Java or Python rather than C [8]. Microworlds such as Karel the Robot [9] make every action visible in a simplified language. The framework combines the two approaches: students write real C, and scaffolding functions emit events that a Python visualiser replays.

Plain-language error messages help novices [10], and automated assessment usually compares final output in a sandbox [11]. Output checks ignore how a student reached the answer, and crashes give little feedback. The C harness therefore reports crashes, timeouts and out-of-bounds errors clearly, keeps the trace up to the point of failure, and compares traces with a reference solution.

Scope. The project will deliver at least one array and one labyrinth task for LZSCC.111, together with the harness, the visualiser, automated tests and a small descriptive user study. It will not deliver a general C visualiser, a grading system or a statistically powered evaluation.

3 Proposed Project

3.1 Aims and Objectives

The aim of the project is to build a reusable framework of visual C tasks of increasing difficulty, aligned with the LZSCC.111 curriculum, and to gather evidence on whether it helps students learn. The objectives are:

O1 – Literature review on program visualisation, scaffolding and novice programming.

O2 – Design the architecture that connects students' C code to Python visualiser through an event-trace format.

O3 – Build a C harness that runs code safely, catching crashes, infinite loops and out-of-bounds errors then reporting them in plain language.

O4 – Implementing (at least 2) tasks with different difficulty, each with C scaffolding, a solution and a visualisation.

O5 – Automated tests for the framework so that new tasks can be added.

O6 – Evaluate the framework technically and through a small user study.

The project addresses two research questions:

RQ1: How can a visual feedback framework for C be designed so that students implement only the core logic, while new tasks remain easy to add?

RQ2: Do students who use the visual tasks report better understanding and engagement than when they use plain text-based exercises?

3.2 Methodology

The project will follow an iterative approach, with a working increment shown to the supervisor every week. This suits the project because each task can be built, tried and adjusted on its own, and early feedback on the first task will shape the design of the rest. A waterfall approach was considered and rejected, since the requirements for later tasks depend on what is learned from the first prototype.

The framework separates student code from visualisation through an event trace. Students solve each task by calling scaffolding functions, such as one that compares two elements or moves the robot, and each function records its action as an event (e.g., "compared index 3"). The C harness writes these events as JSON lines to a separate file, so the student's printf output does

not interfere. The Python visualiser then replays the trace as an animation. Since events are flushed as they occur, a crashing program still leaves a trace up to the point of failure (O3). The trace also lets students step backwards [7] and makes visualisations testable: automated tests compare it against the trace of a reference solution [11](O5).

3.3 Evaluation

The evaluation has two parts, one for each research question. For RQ1, every task must accept its reference solution and catch a set of deliberately faulty solutions, such as an off-by-one error, an infinite loop and an out-of-bounds access. The time and lines of code needed to add a new task will also be measured. For RQ2, participants will solve tasks with and without the visualiser and then complete a questionnaire on understanding and engagement, including the System Usability Scale [12]. Where possible, a short concept quiz will be given before and after the session. Results will be analysed descriptively.

3.4 Participation and ethics

The user study will aim for 10 to 15 participants, current LZSCC.111 students. This size is realistic for the timeframe and enough for a descriptive comparison, though not for statistically significant results. Participants will receive an information sheet and sign a consent form, and all responses will be anonymised. Ethics approval will be requested early in Michaelmas term so that the study can be done in the Lent term. Participation is voluntary and has no effect on LZSCC.111 marks; participants may be withdrawn at any time. Data will be stored securely in line with GDPR and deleted after the project.

4 Programme of Work

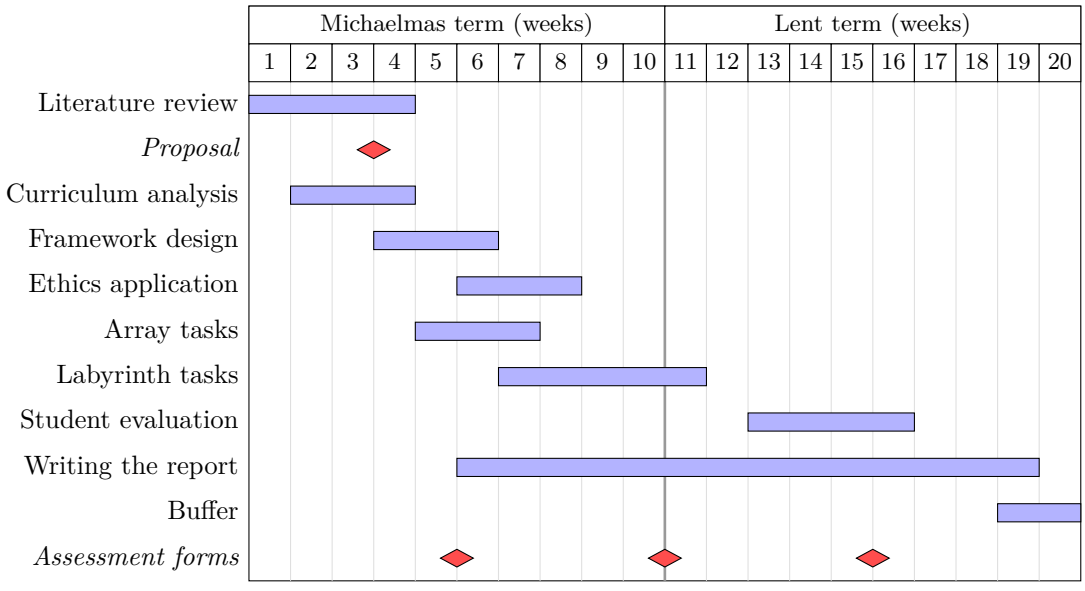


Figure 1: Project schedule. There is a four-week break between weeks 10 and 11.

5 Resources Required

- **Software:** gcc/clang, Python, pygame and Git are all free and open-source, and I will install them myself. A free personal GitHub account (username: florea2005). Overleaf: a free account, or access through Lancaster's email.

- **Hardware:** My personal laptop will be used for development, testing and writing the report. No special hardware or GPU is needed. Lancaster University Leipzig's lab machines will be used during testing to confirm that the tasks run in the real LZSCC.111 environment.

6 Risks and Contingency

Table 1: Risks and mitigation.

Risk	Likelihood	Impact	Mitigation
Too few participants for the study	medium	high	run it during a workshop
C-Python communication is harder than expected	medium	high	build a minimal prototype with only array task first
Illness or other absence	low	medium	Buffer time in the schedule
Ethics approval is delayed	medium	high	run it only with supervisor

References

[1] P. A. Smith and G. I. Webb, 'The efficacy of a low-level program visualisation tool for teaching programming concepts to novice C programmers,' *Journal of Educational Computing Research*, vol. 22, no. 2, 2000. DOI: 10.2190/N0VV-0P48-XJ9G-F8WV

[2] S. Fincher et al., 'Notional machines in computing education: The education of attention,' in *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '20)*, ACM, 2020, pp. 21–50. DOI: 10.1145/3437800.3439202

[3] C. D. Hundhausen, S. A. Douglas and J. T. Stasko, 'A meta-study of algorithm visualisation effectiveness,' *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002. [Online]. Available: https://www.academia.edu/15636659/A_Meta_Study_of_Algorithm_Visualization_Effectiveness

[4] T. L. Naps et al., 'Exploring the role of visualisation and engagement in computer science education,' *ACM SIGCSE Bulletin*, vol. 35, no. 2, pp. 131–152, 2003. DOI: 10.1145/782941.782998

[5] J. Sorva, 'Notional machines and introductory programming education,' *ACM Transactions on Computing Education*, vol. 13, no. 2, 2013. DOI: 10.1145/2483710.2483713

[6] A. Moreno, N. Myller, E. Sutinen and M. Ben-Ari, 'Visualizing programs with Jeliot 3,' in *Proceedings of the Working Conference on Advanced Visual Interfaces (AVI '04)*, ACM, 2004, pp. 373–376. DOI: 10.1145/989863.989928

[7] P. J. Guo, 'Online Python tutor: Embeddable web-based program visualization for CS education,' in *Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE '13)*, ACM, 2013, pp. 579–584. DOI: 10.1145/2445196.2445368

[8] J. Sorva, V. Karavirta and L. Malmi, 'A review of generic program visualisation systems for introductory programming education,' *ACM Transactions on Computing Education*, vol. 13, no. 4, 2013. DOI: 10.1145/2490822

[9] R. E. Pattis, *Karel the Robot: A Gentle Introduction to the Art of Programming*. New York: Wiley, 1981.

[10] B. A. Becker, G. Glanville, R. Iwashima, C. McDonnell, K. Goslin and C. Mooney, 'Effective compiler error message enhancement for novice programming students,' *Computer Science Education*, vol. 26, no. 2–3, pp. 148–175, 2016. DOI: 10.1080/08993408.2016.1225464

[11] P. Ihantola, T. Ahoniemi, V. Karavirta and O. Seppälä, 'Review of recent systems for automatic assessment of programming assignments,' in *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, ACM, 2010, pp. 86–93. DOI: 10.1145/1930464.1930480

[12] J. Brooke, 'SUS: A 'quick and dirty' usability scale,' in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester and I. L. McClelland, Eds., London: Taylor & Francis, 1996, pp. 189–194.